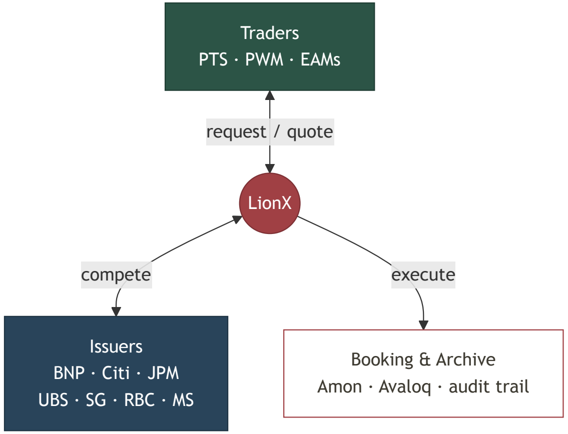
LionX Migration

Replatforming MNY to a cloud-native foundation on AWS.

On-prem today → AWS cloud-native

What is LionX

Pictet's trading platform for structured products (internal name: MNY).



Connect
one protocol to every issuer

Quote
issuers compete, best execution

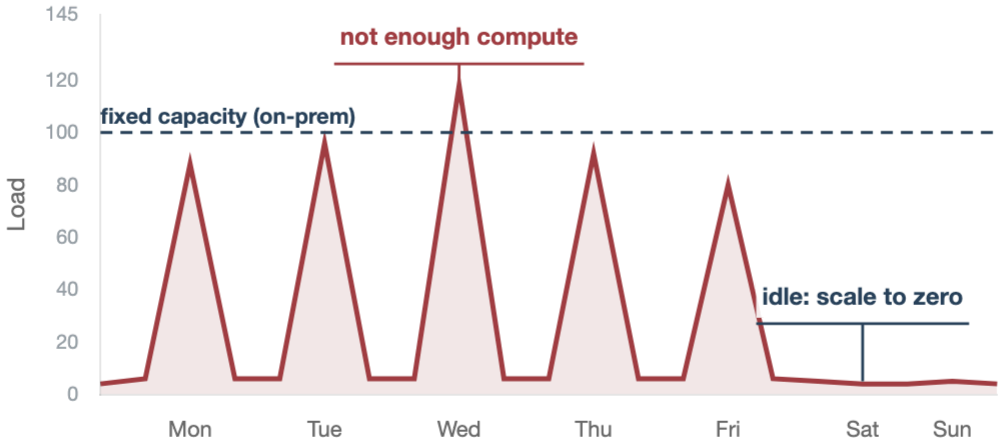
Execute
routed to internal booking (Amon, Avaloq)

Archive
every quote and trade, full audit trail

One interface for traders. One integration point for issuers, across PTS, PWM and EAMs.

Why move to AWS

Our load peaks on market data and sits near zero off-hours and weekends.



Pictet's cloud strategy
LionX aligns with the group's move to AWS

Simplify
on-prem infra back to industry standards

Cloud-native
AWS flexibility, autoscaling on peaks

Full control
over development, operations and infrastructure

Fixed on-prem capacity sits idle most of the week, yet still falls short at peaks. AWS scales up on spikes and down to zero when idle.

Today's infrastructure limits

The on-prem stack works, but it shows its limits. AWS addresses each one.

Single-node MongoDB

reliability and architecture at risk

✓ **Managed, Multi-AZ, resilient and scalable**

No horizontal scaling

several components cannot absorb peaks

✓ **Elastic, scales with demand, no fixed capacity**

No zero-downtime deploys

PROD is stopped for every upgrade

✓ **Blue/green rollouts, zero downtime**

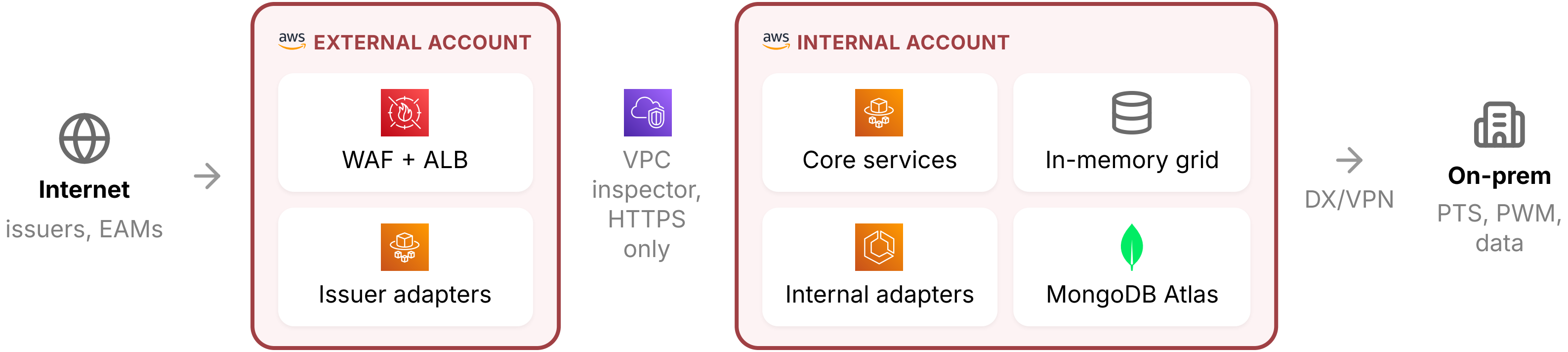
Pipeline to simplify

Puppet / Docker / Kubernetes ⇒ Terraform

✓ **One Terraform IaC, simple CI/CD**

Target architecture, step 1: replatform

Not a lift-and-shift: we retire the hybrid setup for managed services (Atlas, ALB) and a single deployment model.



Issuers and EAMs enter through the external account. PTS and PWM traders reach the core from on-prem, over the private link.

Shared across both accounts: CloudWatch | Secrets Manager | ACM PCA | S3 snapshots

Target architecture, step 2: cloud-native

Same accounts, same flows, same security model as step 1. The refactor changes six things inside them.

	AFTER STEP 1		AFTER STEP 2	
INGRESS	WAF + ALB	→	API Gateway + WAF	throttling and auth at the edge
ADAPTERS	Containers, always on	→	Lambda functions	scale to zero, pay per call
CORE	Stateful, fixed size	→	Stateless, scales out	sized to load, not to peak
STATE	In-memory data grid	→	DocumentDB	state survives restarts
DATABASE	MongoDB Atlas	→	DocumentDB	same API, a fraction of the run cost
INTEGRATION	Point-to-point calls	→	EventBridge	decoupled, replayable events

Everything else carries over from step 1: the two accounts, the VPC inspector, DX/VPN to on-prem, observability.

A two-phase migration

Business continuity first, cloud-native gains next.

PHASE 1: REPLATFORM

Iso-functional, simpler stack
same app behavior, managed services replace the hybrid setup

Parity
performance, security, resilience and audit at least equivalent

Modern operations
single push-based CI/CD, IaC, blue/green deploys

PHASE 2: LEVERAGE THE CLOUD

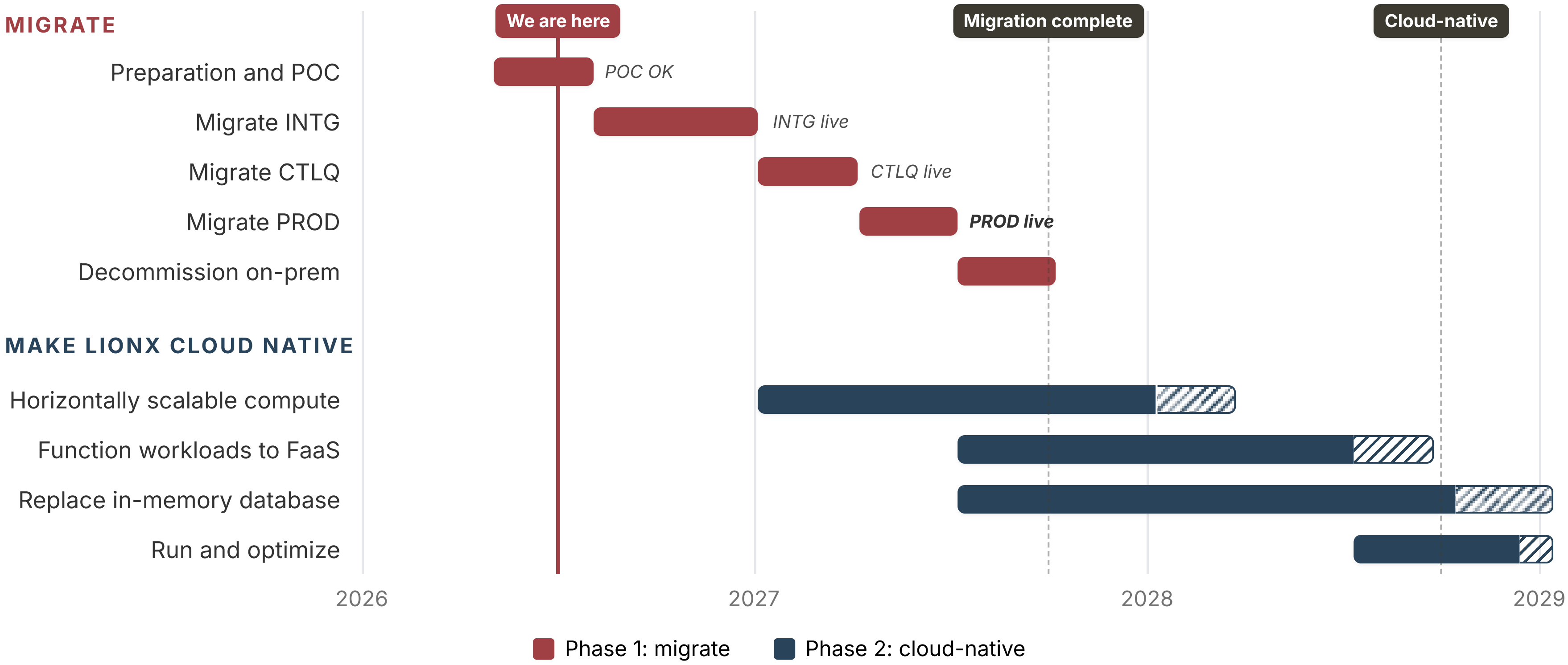
Elasticity
autoscaling on demand, scale down to 20% on nights and weekends

High availability
multi-AZ, self-healing, automated DR and failover tests

Serverless and cloud-native
FaaS adapters on Lambda, DocumentDB, no in-memory database

Roadmap

Migrate first to reach parity, then modernize. Phased from 2026 to 2028.



First environment live by end 2026, production on AWS by mid 2027.

Risks we are managing

Known upfront, with mitigation.

Shared Pictet services

their maturity paces us, possible temporary dual monitoring

Network complexity

on-prem links, issuer connectivity, firewall inspection

Replatforming surprises

different IO, network and timeout behavior in the cloud

Service continuity

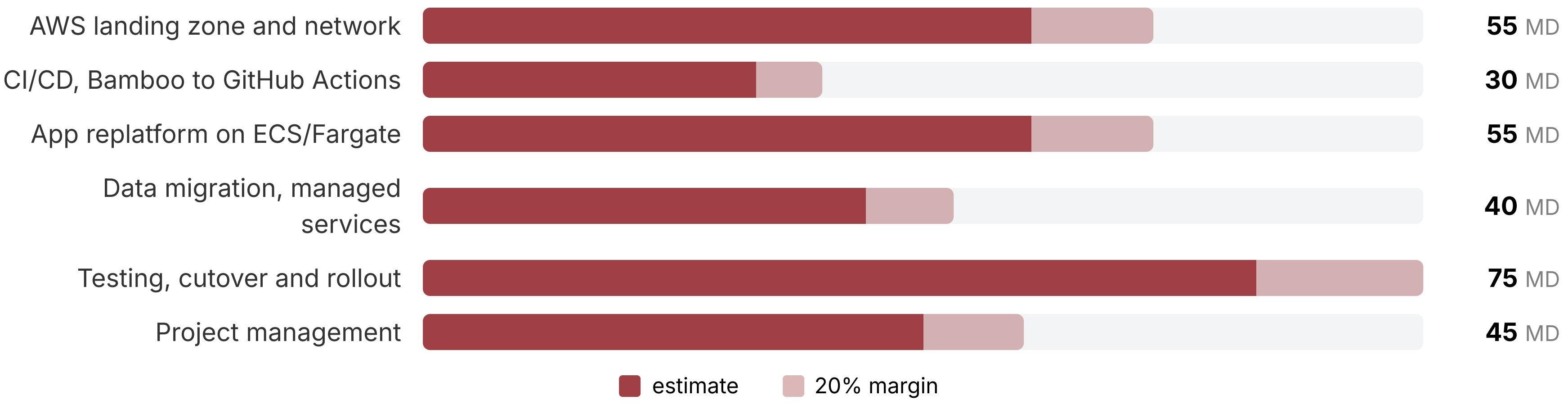
adapting operations and on-call during ramp-up

Each risk has an owner and a mitigation tracked in the migration plan.

1805

Migration effort: Phase 1

One-time effort to replatform onto AWS, CI/CD from Bamboo to GitHub included.

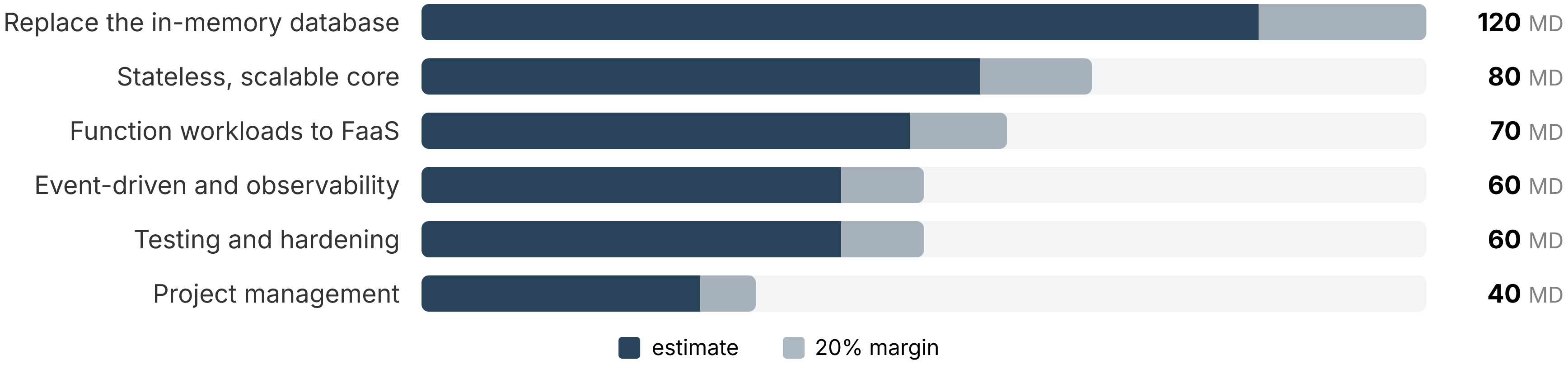


~360
man-days (300 + 20% margin)

~1.6
FTE-year

Migration effort: Phase 2

Development effort for the cloud-native refactor. A rougher estimate than Phase 1, higher uncertainty.

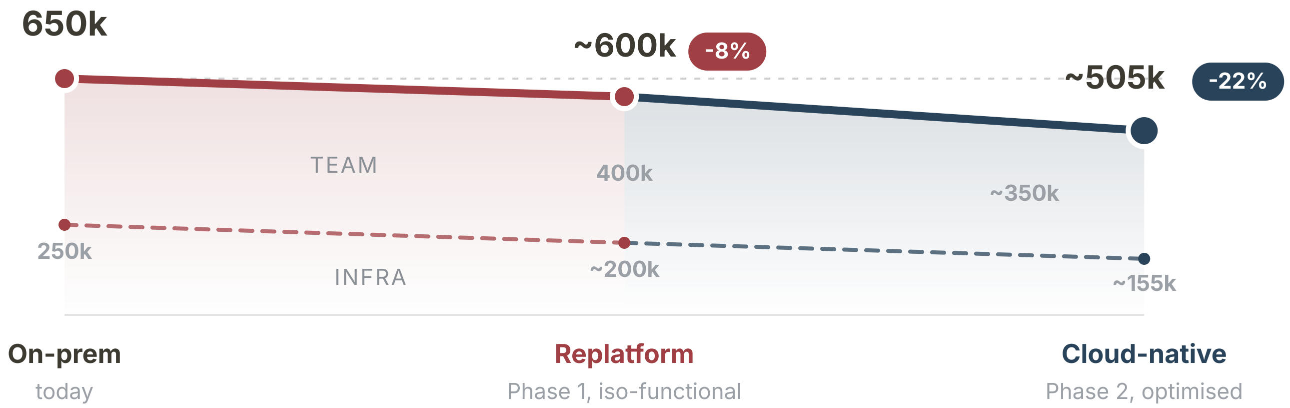


~520
man-days (430 + 20% margin)

~2.3
FTE-year

The cost curve

Run cost, in CHF per year, from Zurich list prices. Atlas through the replatform, DocumentDB after, plus 25% contingency.



ONE-TIME MOVE	BREAK-EVEN	NOT COUNTED
~1.4M CHF	**~9 years**	**Hardware refresh**
876 man-days, margins included	run alone, before the avoided refresh	nor downtime, licences, ops gains past -12%

Next steps

What we are asking for today.

Green-light the plan

approve the two-phase migration and its budget envelope

Chain the ADRs

record the architecture decisions back to back to validate and start fast

Integration env by end 2026

running on AWS inside a real deployment pipeline

Questions and discussion welcome